

Tarea 1

Módulo 7 - Data Pipelines y plataformas de visualización

Claudio Torres Fonseca

mail: claudio.torresf@usm.cl

Diplomado en Data Engineer
2026

1. Contexto del Negocio

Nuestra compañía de Retail ha consolidado su expansión en el mercado latinoamericano operando activamente en tres sedes: **Santiago (Chile)**, **Lima (Perú)** y **Buenos Aires (Argentina)**. Sin embargo, los equipos locales extraen y disponibilizan sus transacciones utilizando tecnologías e infraestructuras completamente heterogéneas.

El Directorio de la empresa requiere un **Dashboard de Ventas Unificado** expresado estrictamente en **Dólares Americanos (USD)**. Este reporte debe actualizarse de forma automatizada todos los días a primera hora de la mañana para la toma de decisiones estratégicas.

Como ingenieros de datos del equipo central, su misión es construir un ecosistema elástico, modular y tolerante a fallos que extraiga, unifique, valide y almacene esta información aplicando prácticas modernas de **ELT**, **FinOps** y **Gobernanza de Datos**.

2. Objetivos del Proyecto

- Desarrollar un pipeline de procesamiento distribuido utilizando **Apache Beam (Python SDK)**.
- Implementar el paradigma de almacenamiento moderno **PSA (Persistent Staging Area)** para garantizar la resiliencia y auditabilidad de los datos.
- Garantizar la calidad de los datos en base a reglas de negocio mediante un **Contrato de Datos (Data Contract)**.
- Orquestar la secuencia lógica de ejecución, manejo de errores y reintentos mediante un **DAG de Apache Airflow**.

3. Arquitectura del Pipeline y Requisitos Técnicos

El proyecto está compuesto estructuralmente por **dos (2) Jobs independientes de Apache Beam**, los cuales serán coordinados cronológicamente por **Apache Airflow**.

Job 1 de Apache Beam: Ingestión y Creación de la Capa PSA

Este primer proceso se enfocará exclusivamente en las fases de Extracción y Carga (E+L) hacia la zona de aterrizaje inmutable (Landing Zone).

- **Lógica del Job:** El script de Beam debe leer las tres fuentes de origen en paralelo, homogeneizarlas bajo un único esquema común y escribir los datos en una estructura de carpetas histórica (PSA) añadiendo la fecha de procesamiento (proc_date=YYYY-MM-DD).
- **Regla FinOps:** El output de este job en la capa PSA debe ser almacenado obligatoriamente en formato **Apache Parquet** para optimizar el espacio de almacenamiento y los futuros costos de escaneo.

Contrato Estructural para la Capa PSA (Homogeneización)

Para asegurar la modularidad del ecosistema, el Job 1 no debe aplicar reglas de negocio ni filtrados de filas. Su única responsabilidad transformacional es mapear las distintas fuentes a un **Esquema Común de Transición** antes de escribir el archivo Parquet unificado. Es obligatorio que el output del Job 1 contenga exactamente las siguientes columnas:

- id_transaccion (String): Identificador único de la venta.
- ciudad (String): Ciudad de origen.
- monto_original (String): El valor monetario bruto. **¡Atención!** Debe guardarse estrictamente como **texto (String)** para permitir el ingreso de datos ruidosos al PSA sin romper la ingesta.
- moneda_origen (String): Código de tres letras inyectado por el script según la procedencia (CLP, PEN o ARS).
- fecha_transaccion (String): Fecha original del evento.
- ingestado_at (String/Timestamp): Marca de tiempo computacional que registre el momento exacto de la ejecución del pipeline.

Job 2 de Apache Beam: Transformación y Validación de Contratos

Este segundo proceso representa la fase de Transformación (T) y consumirá los datos alojados en la capa PSA generada por el Job 1.

- **Contrato de Datos (Data Contract):** Antes de operar el negocio, el Job debe validar que los registros del PSA cumplan las siguientes condiciones:
 - El campo monto_original debe ser estrictamente numérico y mayor a cero (> 0).

- El campo ciudad solo puede contener los valores: 'Santiago', 'Lima' o 'Buenos Aires'.
- Si un registro falla el contrato, debe ser desviado a una cola de registros corruptos (Dead-Letter Queue) en formato JSON para su posterior auditoría, evitando que el pipeline completo falle.
- **Regla de Negocio:** Cruzar los datos aprobados con un archivo maestro de tipos de cambio (tipo_cambio.csv) para convertir los montos locales (CLP, PEN, ARS) a USD según el factor del día.
- **Destino (Gold Layer):** Escribir el resultado final enriquecido y deduplicado en la carpeta Gold, donde quedan disponibles para su uso por la base de datos analítica para el consumo de BI.

Orquestación con Apache Airflow

Deben empaquetar la solución en un archivo DAG de Airflow (dag_orden_ventas_latam.py) que cumpla con los siguientes estándares de producción:

- **Secuencialidad:** El Job 2 de Beam no puede iniciar bajo ninguna circunstancia si el Job 1 no ha finalizado con éxito (Job 1 $\gg$ Job 2).
- **Idempotencia:** El DAG debe parametrizar las fechas de ejecución utilizando los macros nativos de Airflow (`{{ ds }}`). Si el pipeline se ejecuta dos veces para el mismo día de datos, el resultado final no debe duplicarse.
- **Políticas de Alerta y Resiliencia:** Configurar un máximo de 3 reintentos (retries) automáticos con un intervalo de 5 minutos en caso de caídas de infraestructura.

4. Esquemas y Datos de Ejemplo

Fuentes de Entrada (Input Data)

A. Santiago (ventas_santiago.csv) - CSV con encabezados

```
id_transaccion , ciudad , monto , fecha
STGO-001,Santiago,25000,2026-06-10
STGO-002,Santiago,42000,2026-06-10
```

B. Lima (ventas_lima.parquet) - Formato Binario Columnar

Representación visual del contenido del archivo:

transaction_id	city	local_value	sales_date
LIMA-981	Lima	150.50	2026-06-10
LIMA-982	Lima	89.90	2026-06-10

C. Buenos Aires (ventas_buenos_aires.json) - JSON Lines

```
{ "tx_id": "BA-551", "sucursal": "Buenos Aires",  
  "total": 12000, "timestamp": "2026-06-10" }  
{ "tx_id": "BA-552", "sucursal": "BsAs_Erronea",  
  "total": -500, "timestamp": "2026-06-10" }  
{ "tx_id": "BA-553", "sucursal": "Buenos Aires",  
  "total": "NULO_ERROR", "timestamp": "2026-06-10" }
```

D. Tabla Maestra de Tipos de Cambio (tipo_cambio.csv)

```
codigo_moneda,pais,factor_usd  
CLP,Chile,0.0011  
PEN,Peru,0.27  
ARS,Argentina,0.0010
```

Salidas Esperadas (Expected Outputs)

Capa Gold (Datos Consolidados)

Ubicación final: data/gold/proc_date=2026-06-10/ (Formato Parquet):

id_transaccion	ciudad	monto_usd	fecha_compras
STGO-001	Santiago	27.50	2026-06-10
STGO-002	Santiago	46.20	2026-06-10
LIMA-981	Lima	40.635	2026-06-10
LIMA-982	Lima	24.273	2026-06-10
BA-551	Buenos Aires	12.00	2026-06-10

Capa de Aislamiento: Dead-Letter Queue (DLQ):

Ubicación final: data/errors/proc_date=2026-06-10/ (Formato JSON Lines)

```
{ "raw_record": { "tx_id": "BA-552", "sucursal": "BsAs_Erronea",  
  "total": -500, "timestamp": "2026-06-10" },  
  "motivo_rechazo": "Ciudad no autorizada en contrato regional o  
    monto menor a cero" }  
{ "raw_record": { "tx_id": "BA-553", "sucursal": "Buenos Aires",  
  "total": "NULO_ERROR", "timestamp": "2026-06-10" },  
  "motivo_rechazo": "Monto no corresponde a un formato numerico  
    valido (float/int)" }
```

5. Estructura de Entrega

Deberá subir su solución en una carpeta comprimida o un repo Git con la siguiente estructura de archivos:

```
tarea-beam-airflow/
├── dags/
│   └── dag_orden_ventas_latam.py      # Código del DAG de Airflow
├── beam_jobs/
│   ├── job1_ingest_to_psa.py         # Script Apache Beam (Ingesta)
│   └── job2_transform_gold.py        # Script Apache Beam (Transformación)
└── README.md                        # Documentación técnica y análisis FinOps
```

6. Rúbrica de Evaluación

6.1. Desarrollo e Infraestructura en Apache Beam (3p.)

- 3 puntos: Configura de manera óptima los conectores I/O distribuidos para los tres formatos (csv, parquet, json). Utiliza herencia correcta de DoFn u operadores Map/FlatMap para la homogeneización. Implementa de forma impecable los Side Inputs de Beam para el cruce con el archivo de tipo de cambio.
- 1-2,5 puntos: El pipeline funciona pero presenta ineficiencias, como cargar colecciones completas en memoria de los workers sin usar Side Inputs distribuidos o mezclar lógicas de transformación dentro de la etapa de ingesta (violando el principio del PSA).
- 0-1 puntos: El script de Beam no compila, confunde el uso de PCollection con listas nativas de Python o no procesa las fuentes en paralelo.

6.2. Orquestación, Secuencialidad e Idempotencia en Airflow (1,5p.)

- 1,5 punto: Construye un DAG con dependencias explícitas (Job 1 $\gg$ Job 2). Implementa parametrización dinámica del contexto utilizando macros de Airflow (ds) tanto para las rutas de lectura como de escritura, garantizando que el pipeline sea 100 % ejecutable de manera retrospectiva (backfill) e idempotente.
- 0-1 puntos: El código del DAG arroja errores de sintaxis en Airflow, no define dependencias lógicas o las tareas se ejecutan en desorden.

6.3. Gobernanza de Datos y Manejo de Errores (Data Contracts & DLQ) (1p.)

- 1 punto: Implementa con éxito el contrato estructural en el Job 1 (guardando el monto bruto como String) y el contrato de calidad en el Job 2. Separa los registros válidos de los corruptos utilizando salidas etiquetadas (Tagged Outputs) hacia la Dead-Letter Queue sin botar el proceso.
- 0-0,5: Aplica validaciones básicas de datos mediante bloques try/except, pero descarta las filas erróneas (pérdida de datos) en lugar de aislarlas en la ruta analítica de la DLQ solicitada, o realiza el cast numérico antes de tiempo destruyendo la fidelidad del PSA.
- 0 puntos: No valida estructuras ni tipos de datos. El pipeline completo se cae al procesar los registros con errores.

6.4. Documentación (0,5 p.)

- 0,5 puntos: El archivo README.md entrega una descripción del proyecto, indicando una descripción de cada script Python, incluyendo los parámetros necesarios para su ejecución.
- 0 puntos: no se incluye un README.md o su contenido no refleja el contenido de los archivos python solicitados.

Consideraciones

- Se descontará 1 punto por día de atraso en la entrega de la tarea. Debe asegurarse que la versión correcta de su tarea sea subida a la plataforma, cualquier modificación posterior a la fecha de entrega será considerado como atraso en la entrega.
- Se descontará 1 punto por cada error de sintaxis o error en tiempo de ejecución no manejado. Para este punto se considerará el uso de las instrucciones en el archivo Readme.
- Está permitido utilizar Inteligencias Artificiales (IA) para el desarrollo de la tarea, pero debe agregar el prompt utilizado como Anexo. En caso de no incluir el Anexo y si se detecta el uso de una IA se considerará su tarea como copia con nota mínima.

Fecha entrega: 22 Junio 2026